

설정과 커스터마이징

Codex를 우리 팀의 도구로 만들기

One agent for everywhere you code - CLI, IDE, App, Web

Codex Deep Dive Workshop (2-Day) - Module 6 / Configuration

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



MODULE 06

설정과 커스터마이징

Codex를 우리 팀의 도구로 만들기



이 모듈의 목표

이 모듈에서 얻어갈 네 가지

GOAL

이 모듈을 마치면 설정 우선순위를 이해하고, 팀 표준 config를 작성하며, AGENTS.md로 에이전트를 지도하고, MCP로 도구를 안전하게 연결할 수 있습니다.

LAYERS

계층 이해

6단계 우선순위로 설정 출처를 파악합니다

CONFIG

팀 표준

config.toml과 프로파일로 표준을 정의합니다

AGENTS

에이전트 지도

AGENTS.md로 프로젝트 지침을 전달합니다

MCP

안전한 확장

MCP 서버를 보안 원칙에 맞게 연결합니다

70 min

강의 55 + 랩 15

6

설정 계층

TOML

config 형식

Lab 5

프로파일 작성

설정 계층 개요

여섯 단계 우선순위 체계

LAYERED CONFIG

Codex 설정은 여섯 계층입니다. 내장 기본값부터 CLI 플래그까지, 뒤 계층이 앞 계층을 덮어씁니다.
적용 값이 헛갈리면 `/debug-config`로 출처를 확인합니다.

1 Defaults

내장 기본값

2 System

`/etc/codex`

3 User

`~/.codex`

4 Project

`.codex` 커밋

5 Profile

`--profile`

6 CLI

최종 우선

MENTAL MODEL CSS 특정도처럼 생각하십시오.

개인 config는 안전망, 프로젝트 config는 팀 합의, CLI 플래그는 그 순간의 예외입니다.

6 Levels

우선순위 체계

User

가장 자주 편집

Project

레포 커밋, 팀 공유

`/debug-config`

적용값과 출처 확인

Config Basics

config.toml 기본 구조

~/.codex/config.toml

```
# 최상위 키로 기본 동작을 정의
model = "gpt-5.5-codex"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
# 섹션으로 영역별 설정을 그룹화
[tui]
theme = "dark"
```

TOML FORMAT config는 TOML 형식입니다. 최상위 키로 기본 동작을 정하고, 대괄호 섹션으로 영역별 설정을 그룹화합니다.

TOML

설정 형식

Top keys

기본 동작

[section]

영역별 그룹

~/.codex

사용자 위치

Advanced Config

프로파일로 설정 묶음 전환

~/.codex/config.toml - profiles

```
# 프로파일, 작업별 설정 묶음
[profiles.fast]
model = "gpt-5.1-codex-mini"
model_reasoning_effort = "low"
[profiles.deep]
model_reasoning_effort = "high"
approval_policy = "on-request"
```

PROFILES 작업 유형별 설정 묶음을 정의하고, `codex --profile fast` 처럼 빠르게 전환합니다.
개인 작업과 클라이언트 작업의 안전 수준을 분리할 때 특히 유용합니다.

fast

빠른 작업 묶음

deep

깊은 작업 묶음

--profile

즉시 전환

분리

작업별 안전 수준

Config Reference

주요 설정 키

01	model	사용할 기본 모델	코어
02	model_reasoning_effort	추론 강도 (low/medium/high)	코어
03	approval_policy	승인 정책 (untrusted/on-request/never)	안전
04	sandbox_mode	샌드박스 (read-only/workspace-write)	안전
05	shell_environment_policy	셸 환경 변수 노출 제어 (시크릿 차단)	보안
06	mcp_servers.* / profiles.*	MCP 서버 등록, 작업별 프로파일	확장
07	[features]	skills, undo 등 기능 플래그	기능

Core

model, effort

Safety

approval, sandbox

Secret

shell policy

Extend

mcp, profiles, features

Sample Config

팀 표준 설정 예시

~/.codex/config.toml - team standard

```
# 팀 표준 config.toml 예시
model = "gpt-5.5-codex"
model_reasoning_effort = "medium"
approval_policy = "on-request"
sandbox_mode = "workspace-write"

[mcp_servers.github]
command = "npx"
args = ["-y", "@modelcontextprotocol/server-github"]
```

PRACTICAL 팀 표준은 안전한 기본값을 전역에 두고 프로젝트별로 오버라이드하는 구조가 좋습니다.
개인 config는 안전망, 프로젝트 config는 팀 합의입니다.

medium

균형 잡힌 강도

on-request

권장 승인

github

팀 공용 MCP

공유

프로젝트 커밋

Environment Variables

환경 변수로 오버라이드

01	OPENAI_API_KEY	API 키 인증 (SDK, CI)	인증
02	CODEX_HOME	설정과 상태 디렉토리 위치	환경
03	CODEX_MODEL	기본 모델 오버라이드	모델
04	HTTP_PROXY / HTTPS_PROXY	사내 프록시 환경 지원	네트워크

PRECEDENCE 환경 변수는 config 파일보다 우선하는 경우가 많아 CI나 일시적 오버라이드에 유용합니다.
사내 프록시 환경에서는 프록시 변수 설정이 인증 실패를 막는 첫 단계입니다.

API_KEY
CI 인증 표준

CODEX_HOME
디렉토리 변경

PROXY
사내망 지원

우선
config보다 위

Shell Environment Policy

시크릿을 에이전트에게서 격리

~/.codex/config.toml - shell policy

```
# 기본값은 셸 환경 변수 전체 상속, 시크릿을 차단하세요
[shell_environment_policy]
policy = "exclude"
exclude = ["AWS_SECRET_ACCESS_KEY", "AWS_SESSION_TOKEN",
           "GITHUB_TOKEN", "NPM_TOKEN"]

# 또는 화이트리스트(include_only), 샌드박스 전용 변수(set) 사용
```

WHY IT MATTERS 에이전트가 디버그 출력이나 커밋에 자격 증명을 실수로 포함하는 사고를 원천 차단합니다.
필요해지기 전에 미리 설정해 두십시오.

exclude

시크릿 블랙리스트

include_only

화이트리스트

set

샌드박스 전용 변수

사전 설정

사고 전에 미리

Permissions

권한과 승인 정책 복습

UNTRUSTED

신뢰 안 함

신뢰되지 않은 모든 동작에 승인을 요구합니다

ON-REQUEST

요청 시

에이전트가 필요하다고 판단할 때 묻습니다

ON-FAILURE

실패 시

샌드박스에서 막힌 경우에만 묻습니다

NEVER

묻지 않음

승인 없이 진행, 격리 환경에서만 권장합니다

RECOMMENDED 권한은 안전과 생산성의 균형입니다.

일상 개발은 on-request + workspace-write, 코드 리뷰는 untrusted + read-only, CI는 never + workspace-write (격리 필수)입니다.

Daily

on-request

Review

untrusted

CI

never

균형

안전과 생산성

Speed

응답 속도 최적화

FASTER

속도를 높이는 법

- reasoning effort를 작업에 맞게 낮추기
- 컨텍스트를 핵심 파일로 제한
- 사전 빌드와 의존성 캐시 활용
- 불필요한 도구 호출 줄이기

CAUTION

주의할 점

- 지나친 effort 절감은 정확도 저하
- 컨텍스트 부족은 잘못된 가정 유발
- 속도와 품질의 균형점을 작업별로 탐색
- 반복 작업은 워크플로우로 표준화

BALANCE 속도는 공짜가 아닙니다. effort를 너무 낮추거나 컨텍스트를 너무 줄이면 품질이 떨어집니다. 작업별로 균형점을 찾는 것이 핵심입니다.

Effort

강도 조절

Context

핵심만 참조

Cache

준비 가속

균형점

작업별 탐색

Rules

동작 규칙 정의

RULES

Rules는 에이전트가 항상 따라야 할 제약과 지침을 정의합니다. 코딩 컨벤션, 금지 사항, 우선순위 등을 명시합니다.

EXAMPLES

규칙 예시

- 코드 스타일과 컨벤션 강제 (예: TypeScript strict 모드, 함수형 컴포넌트 우선)
- 특정 라이브러리나 패턴의 사용 또는 금지
- 커밋 메시지와 PR 형식 규정 (예: conventional commits)
- 보안 민감 영역의 변경 제한

WHERE Rules는 AGENTS.md에 자연어로 적거나 config의 request_rule 기능으로 적용합니다.

다음 슬라이드부터 AGENTS.md를 자세히 다룹니다.

Style

컨벤션 강제

Forbid

패턴 금지

Format

커밋, PR 규정

Protect

민감 영역 제한

Hooks

이벤트 기반 확장

HOOKS

Hooks는 에이전트 동작의 특정 시점에 사용자 정의 스크립트를 실행합니다. 검증, 알림, 정책 적용 등을 자동화합니다.

1 Pre-action

동작 전 검사

2 Action

에이전트 실행

3 Post-action

동작 후 처리

4 Notify

결과 알림

USE CASES 동작 전 검증, 동작 후 자동 테스트, 외부 시스템 알림 같은 결정론적 워크플로우 제어에 씁니다.
승인 모드가 사람의 게이트라면 Hooks는 자동화된 게이트입니다.

Event

이벤트 기반

Custom

사용자 스크립트

Validate

검증 자동화

Integrate

외부 연동

AGENTS.md

에이전트를 위한 프로젝트 안내서

AGENTS.MD

프로젝트 루트의 AGENTS.md는 에이전트에게 구조, 컨벤션, 명령, 주의사항을 알려주는 표준 안내 파일입니다. 전역에서 루트, 하위 디렉토리 순으로 이어 읽힙니다 (기본 32KiB 한도).

STRUCTURE

구조 설명

디렉토리 구성과 주요 모듈을 안내합니다

COMMANDS

명령

빌드, 테스트, 린트 명령을 명시합니다

STYLE

컨벤션

코드 스타일과 패턴 규칙을 전달합니다

CAUTION

주의사항

건드리면 안 되는 영역을 경고합니다

Root

프로젝트 안내

Open fmt

다른 도구도 읽음

32 KiB

기본 결합 한도

Cascade

루트에서 하위로

AGENTS.md 예시

실제 작성 형태

프로젝트 루트 / AGENTS.md

```
## Build & Test
- Install: npm ci
- Test: npm test (커밋 전 반드시 통과)
- Lint: npm run lint

## Conventions
- Use TypeScript strict mode
- Prefer functional components

## Do not touch
- /legacy 디렉토리는 수정 금지
```

READABLE 마크다운 섹션으로 명령, 컨벤션, 금지 영역을 명확히 나눕니다. 사람도 읽기 쉽고 에이전트도 따르기 쉬운 형태입니다.

Build

명령 섹션

Style

컨벤션 섹션

Forbid

금지 섹션

MD

사람도 읽기 쉬움

AGENTS.md 베스트 프랙티스

효과적인 지침 작성법

DO

권장 사항

- 실행 가능한 명령을 구체적으로 명시
- 추상적 표현보다 명확한 규칙으로 작성
- 자주 발생하는 실수를 미리 경고
- 전역과 루트, 하위가 연결되며 가까운 파일 우선
- 모노레포는 디렉토리별로 분리

DON'T

피할 것

- 너무 길어 핵심이 묻히지 않게 간결히
- 모순되는 규칙으로 혼란 주지 않기
- 민감 정보나 시크릿을 적지 않기
- 팀 합의 없이 임의 규칙 남발 금지
- 프로젝트 변화에 맞춰 업데이트 누락 주의

OVERRIDE TRICK 양보 불가능한 개인 규칙은 ~/.codex/AGENTS.override.md에 둡니다.
override 파일은 같은 위치의 일반 AGENTS.md보다 항상 우선합니다.

구체적

실행 가능 명령

간결

1페이지 지향

가까운 파일

하위 우선

override

양보 불가 규칙

MCP

Model Context Protocol

MCP

MCP는 에이전트가 외부 도구와 데이터 소스에 표준 방식으로 연결하게 하는 개방형 프로토콜입니다.
Codex는 MCP 서버를 통해 능력을 확장합니다.

STANDARD

표준 프로토콜

도구 연결을 위한 개방형 표준
입니다

TOOLS

도구 제공

서버가 호출 가능한 도구를 노
출합니다

DATA

데이터 접근

외부 리소스를 컨텍스트로 제
공합니다

ECOSYSTEM

생태계

GitHub, DB 등 다양한 서버가
존재합니다

STUDIO

로컬 프로세스

HTTP

원격 서버

Open

개방형 표준

확장

능력 확장 통로

MCP 서버 설정

config.toml에 서버 등록

~/.codex/config.toml - MCP servers

```
[mcp_servers.github]
command = "npx"
args    = ["-y", "@modelcontextprotocol/server-github"]
env     = { GITHUB_TOKEN = "..." }
```

```
[mcp_servers.postgres]
command = "mcp-server-postgres"
args    = ["--connection-string", "$DB_URL"]
```

TIP codex mcp add 한 줄로도 등록되고, OAuth 서버는 codex mcp login을 씁니다.
enabled_tools와 disabled_tools로 서버가 노출하는 도구를 선별하십시오.

mcp add

한 줄 등록

mcp login

OAuth 인증

enabled_tools

도구 화이트리스트

env

시크릿 주입

MCP 보안 고려사항

안전하게 도구 연결하기

PRINCIPLES

보안 원칙

- 신뢰할 수 있는 MCP 서버만 등록
- 서버에 부여하는 권한을 최소화
- 시크릿은 환경 변수로 안전하게 주입
- 서버가 반환한 콘텐츠의 인젝션 경계

MITIGATION

위험 완화

- 외부 데이터의 악성 지시는 데이터로 취급
- 민감 작업은 사람 승인 단계 유지
- 서버 출처와 무결성을 검증
- 접근 로그로 사후 감사 가능하게

KEY RISK 가장 큰 위험은 프롬프트 인젝션입니다.

MCP 서버가 가져온 외부 콘텐츠에 섞인 지시는 명령이 아니라 데이터로 취급해야 합니다. Module 8에서 더 깊이 다룹니다.

Trust

검증된 서버만

Least-priv

최소 권한

Injection

외부 콘텐츠 경계

Audit

접근 로그

Module 6 요약와 Mini Lab 5

핵심 정리와 15분 실습

SUMMARY

설정은 6계층 우선순위 구조이며, config.toml로 표준을, AGENTS.md로 지침을, MCP로 도구 확장을 정의하고, Shell Policy로 시크릿을 격리합니다.

Terminal - Mini Lab 5 (15분)

```
# Step 1. config 열기 Step 2. fast/deep 프로파일 추가
$ ${EDITOR:-vi} ~/.codex/config.toml
# Step 3. 프로파일로 실행해 차이 비교
$ codex --profile fast "Explain src/ structure"
$ codex --profile deep "Find subtle bugs in auth flow"
```

CHECKPOINT 프로파일 2개 저장, --profile 전환 동작, fast와 deep의 속도와 깊이 차이 확인이 통과 기준입니다.

6 Layers

설정 계층

AGENTS.md

에이전트 지침

15 min

프로파일 실습

Next

Plugins, Skills

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

